GrokRxiv:2026.03.ai-os-synthesis [cs.AI] 10 Mar 2026

The AI Operating System

Part V: Modular Synthesis — Composing Four Subsystems into a Unified Runtime

An Elixir/OTP Umbrella Application

Matthew Long
The YonedaAI Collaboration
YonedaAI Research Collective
Chicago, IL
matthew@yoneda.ai

March 10, 2026

Abstract

Parts I–IV of *The AI Operating System* developed four independent subsystems: an Agent Scheduler for process management, a Tool Interface for capability-based access control, a Memory Layer for typed persistent cognition, and a Planner Engine for market-based orchestration. Each subsystem was designed as a self-contained Elixir/OTP application with its own supervision tree, public API, and type specifications.

This paper addresses the synthesis question: *how do these four modules compose into a single, coherent operating system for AI agents?*

We present `AgentOS`, an Elixir umbrella application that composes the four subsystems through (1) a dependency-ordered supervision tree, (2) message-passing interfaces between subsystem boundaries, (3) pipeline composition across layers, and (4) a unified facade module that exposes the complete job lifecycle. We show that every pairwise combination of subsystems yields emergent capabilities: agents invoke tools via capabilities, agents persist state through memory, tool results are cached in typed memory, and the planner orchestrates all three through market mechanisms. The result is a production-grade system demonstrated through the Agent-Hero marketplace—a Next.js + Supabase + Stripe platform where clients post jobs, the planner decomposes and matches them to agents via an order book, agents execute with tool access and memory persistence, and evaluators score outcomes to update reputation.

The implementation comprises 24 Elixir source files totaling approximately 3,500 lines of code, running on the BEAM virtual machine. All composition is achieved through standard functional programming patterns: module boundaries, supervision hierarchies, typed message passing, behaviour callbacks, and pipeline composition.

Keywords: AI operating systems, modular composition, Elixir/OTP, agent orchestration, supervision trees, capability-based security, typed memory, market mechanisms

Contents

1 Introduction **6**
1.1 The Synthesis Problem . . . 6
1.2 Approach: Functional Composition on the BEAM . . . 6
1.3 Mathematical Foundations . . . 7
1.4 Paper Outline . . . 7

2 The Four Subsystems **7**
2.1 Agent Scheduler . . . 8
2.1.1 Supervision Tree . . . 8
2.1.2 Key Modules . . . 8
2.1.3 Public Interface . . . 9
2.2 Tool Interface . . . 9
2.2.1 Three-Tier Registry . . . 9
2.2.2 Capability Tokens . . . 9
2.2.3 Sandboxed Execution . . . 10
2.2.4 Audit Logging . . . 10
2.2.5 Public Interface . . . 10
2.3 Memory Layer . . . 11
2.3.1 Schema Type System . . . 11
2.3.2 Dual-Layer Storage . . . 11
2.3.3 Versioning and Graph . . . 12
2.3.4 Public Interface . . . 12
2.4 Planner Engine . . . 12
2.4.1 Order Book . . . 13
2.4.2 Escrow . . . 13
2.4.3 Job Decomposition . . . 13
2.4.4 Revenue Distribution . . . 14
2.4.5 Reputation and Anti-Gaming . . . 14

3 The Umbrella Application **14**
3.1 Mix Project Configuration . . . 15
3.2 Startup Ordering . . . 15
3.2.1 Why This Order? . . . 16
3.2.2 The one_for_one Strategy . . . 17
3.3 The Unified Facade . . . 17

4 Pairwise Compositions **18**
4.1 Agent Scheduler + Tool Interface . . . 18
4.2 Agent Scheduler + Memory Layer . . . 18
4.3 Tool Interface + Memory Layer . . . 19
4.4 Planner Engine + Agent Scheduler . . . 20
4.5 Planner Engine + Memory Layer . . . 21
4.6 Planner Engine + Tool Interface . . . 21

5 Full Four-Way Composition **22**
5.1 The Complete Job Lifecycle . . . 22
5.2 Step-by-Step Walkthrough . . . 22
5.2.1 Step 1–2: Job Submission and Decomposition (Planner Engine) . 22
5.2.2 Step 3–5: Order Book Matching and Market Clearing (Planner Engine) . . . 23
5.2.3 Step 6: Agent Assignment (Scheduler) . . . 23
5.2.4 Step 7–8: Tool Invocation with Memory Persistence . . . 24
5.2.5 Step 9: Evaluation (Scheduler) . . . 25
5.2.6 Step 10: Settlement and Reputation (Planner Engine) . . . 25
5.3 Pipeline Composition Across Layers . . . 25

6 Production System: Agent-Hero Marketplace **26**
6.1 Technology Stack . . . 26
6.2 Mapping AgentOS to Production . . . 27
6.3 Job Lifecycle in Production . . . 27
6.4 Deployment Architecture . . . 28

7 System Properties **28**
7.1 Fault Isolation . . . 28
7.2 Agent-Level Fault Isolation . . . 29
7.3 Type Safety Across Boundaries . . . 29
7.4 Resource Fairness . . . 30
7.5 Financial Conservation . . . 30

8 Implementation Summary **31**
8.1 Source Code Structure . . . 31
8.2 Key Design Patterns . . . 31

9 Composition Algebra **32**
9.1 Module Interfaces as Types . . . 32
9.2 Composition as Interface Products . . . 32
9.3 The Full Product . . . 32

10 Emergent Properties of Composition **32**
10.1 End-to-End Durable Execution . . . 33
10.2 Self-Improving Reputation . . . 33
10.3 Adaptive Pipeline Composition . . . 33

11 Related Work **33**
11.1 Agent Frameworks . . . 33
11.2 Operating System Analogies . . . 33
11.3 OTP and Supervision . . . 34
11.4 Market Mechanisms for AI . . . 34

12 Conclusion **34**
12.1 Future Work . . . 35

A Module Dependency Matrix **35**

B Supervision Tree Diagram **36**

C Message Flow Diagram **36**

D Algorithm: Complete Job Execution **37**

1 Introduction

The preceding four papers in this series each addressed a single concern of an AI operating system:

Part I: **Agent Scheduler** — Process lifecycle management for AI agents, including credit-weighted scheduling, durable execution with memoization, streaming pipelines, and 6-dimensional evaluation.

Part II: **Tool Interface** — Capability-based access control with a 3-tier registry (builtin, sandbox, MCP), sandboxed execution via isolated BEAM processes, and per-invocation audit logging.

Part III: **Memory Layer** — A typed filesystem for persistent agent cognition, with 24 schema types across 6 subcategories, versioned evolution with vector clocks, graph relationships, and multi-backend storage.

Part IV: **Planner Engine** — Market-based orchestration with an order book matching engine, escrow-backed financial transactions, DAG-based job decomposition, and reputation scoring with anti-gaming detection.

Each part was designed as an independent Elixir/OTP application—a self-contained module with its own supervision tree, public API, type specifications, and test suite. This modularity is not accidental; it is the central architectural principle.

1.1 The Synthesis Problem

The challenge addressed by this paper is *composition*: given four well-defined modules, how do we assemble them into a system that is greater than the sum of its parts? Specifically:

- What is the correct startup ordering, and why does it matter for fault tolerance?
- How do subsystems communicate across module boundaries without violating encapsulation?
- What emergent capabilities arise from pairwise and higher-order compositions?
- How does the composed system support a complete production job lifecycle?

1.2 Approach: Functional Composition on the BEAM

Our approach is grounded in functional programming on the BEAM virtual machine. The composition mechanisms are:

1. **Supervision trees** — OTP supervisors provide fault-isolated startup ordering with automatic restart on failure.
2. **Message passing** — Subsystems communicate via `GenServer.call/cast` across process boundaries, with no shared mutable state.

3. **Behaviour callbacks** — Elixir behaviours define type-safe interfaces that subsystems implement, enabling polymorphic dispatch.
4. **Pipeline composition** — Multi-stage streaming pipelines compose agents, tools, and memory into end-to-end execution flows.
5. **Module facades** — A top-level `AgentOS` module provides a unified API that delegates to the appropriate subsystem.

None of these mechanisms require exotic abstractions. They are the standard tools of Erlang/OTP, refined over 40 years of production use in telecommunications, banking, and messaging infrastructure.

1.3 Mathematical Foundations

The design of AgentOS is informed by ideas from category theory, which provides a language for describing composition, interfaces, and structure preservation. In particular:

- Each subsystem can be understood as an object in a module category, with its public API defining the morphisms available to other subsystems.
- Composition of subsystems corresponds to constructing products and coproducts in this category: the umbrella application is a product of its component applications, while the storage router is a coproduct of backends.
- The type-safe interfaces between subsystems (behaviours, typespecs, message protocols) act as natural transformations that preserve structure across module boundaries.

However, the implementation does not require the reader to be fluent in categorical language. Throughout this paper, we present all constructions concretely in terms of Elixir modules, supervision trees, and message-passing protocols. The categorical perspective serves as background context for readers who find it illuminating, not as a prerequisite for understanding the system.

1.4 Paper Outline

Section 2 reviews the four subsystems and their public interfaces. Section 3 presents the umbrella application and its supervision tree. Section 4 analyzes the six pairwise compositions and their emergent capabilities. Section 5 shows the full four-way composition through a complete job lifecycle. Section 6 describes the production deployment in the Agent-Hero marketplace. Section 7 establishes key system properties: fault isolation, type safety, and resource fairness. Section 11 surveys related work, and Section 12 concludes.

2 The Four Subsystems

Before examining their composition, we summarize each subsystem's architecture, supervision tree, and public API. The goal is to establish the *interfaces* through which composition occurs.

2.1 Agent Scheduler

The Agent Scheduler manages the lifecycle of AI agents as supervised BEAM processes. Its architecture mirrors classical OS process management, adapted for AI workloads.

2.1.1 Supervision Tree

```
AgentScheduler.AppSupervisor (rest_for_one)
  +-- Registry (unique, :agent_registry)
  +-- AgentScheduler.Evaluator (GenServer)
  +-- AgentScheduler.Scheduler (GenServer)
  +-- AgentScheduler.Pipeline (GenServer)
  +-- AgentScheduler.Supervisor (DynamicSupervisor)
      +-- Agent "agent_001"
      +-- Agent "agent_002"
      +-- Agent "agent_n"
```

Listing 1: Agent Scheduler supervision tree

The `:rest_for_one` strategy ensures that if the Registry crashes, all dependent components restart in order. The DynamicSupervisor uses `:one_for_one` so individual agent failures are isolated.

2.1.2 Key Modules

`AgentScheduler.Agent`
: A GenServer managing an individual agent's state machine: `:pending` → `:running` → `:waiting_approval` → `:completed`. Implements durable execution via a memoization store: each step function is keyed by a unique identifier, and completed steps are replayed from the memo store on crash recovery without re-execution.

`AgentScheduler.Scheduler`
: A credit-weighted priority scheduler using Erlang's `:gb_trees` (balanced binary trees) for $O(\log n)$ enqueue/dequeue. Each client accumulates *agent virtual runtime* (avruntime) proportional to credits consumed:

$$\text{avruntime}(c) \mathrel{+}= \frac{\kappa_A \cdot \text{cr}_0}{\text{cr}_c \cdot w(p_c)} \tag{1}$$

where κ_A is agent cost, $\text{cr}_0 = 1000$ is the reference credit amount, cr_c is the client's remaining credits, and $w(p_c)$ is the priority weight (4.0 for contracted, 1.0 for marketplace). The scheduler always dispatches the client with the smallest avruntime.

`AgentScheduler.Pipeline`
: Streaming pipelines with EventEmitter-style pub/sub. Each pipeline consists of ordered stages declaring what event types they publish and subscribe to. The pipeline constraint ensures stages only subscribe to events from earlier stages: $\text{sub}(s_i) \subseteq \bigcup_{j<i} \text{pub}(s_j)$.

`AgentScheduler.Evaluator`
: Six-dimensional quality evaluation across: quality, adherence, speed, cost efficiency, error rate, and revision count. The composite score is a weighted inner product, and reputation is computed as an EWMA: $R_t = \alpha \cdot s_t + (1-\alpha) \cdot R_{t-1}$ with $\alpha = 0.3$.

2.1.3 Public Interface

```
AgentScheduler.submit_job(client_id, job, opts)
  :: {:ok, job_id} | {:error, term()}

AgentScheduler.start_agent(agent_id, profile, opts)
  :: {:ok, pid()} | {:error, term()}

AgentScheduler.create_pipeline(name, stages)
  :: {:ok, pipeline_id} | {:error, term()}

AgentScheduler.get_evaluation(agent_id)
  :: {:ok, map()} | {:error, :not_found}
```

Listing 2: Agent Scheduler public API

2.2 Tool Interface

The Tool Interface mediates between agents and external systems, analogous to device drivers in a classical operating system.

2.2.1 Three-Tier Registry

Tools are organized into three trust tiers:

Table 1: Tool tiers in the 3-tier registry

Tier	Trust Level	Examples
Builtin	Static, trusted	`web-search`, `text-transform`, `pdf-parse`
Sandbox	Isolated execution	`code-exec`, `shell-exec`, `git-clone`
MCP	Runtime-discovered	`server__tool` (namespaced)

The registry supports *freezing*: once frozen, no new tools can be registered. This implements contract-locked configuration where the tool set is immutable during agent execution.

2.2.2 Capability Tokens

Access control is capability-based. Each agent holds unforgeable tokens that grant specific permissions on specific tools:

```
%ToolInterface.Capability{
  agent_id: "agent-1",
  tool_id: "web-search",
  permissions: [:invoke, :inspect],
  rate_limit: 60,            # invocations per minute
  expires_at: ~U[...],
  signature: <<hmac_sha256>>  # tamper detection
}
```

Listing 3: Capability token structure

Authorization checks: (1) tool ID matches, (2) token not expired, (3) HMAC signature valid, (4) `:invoke` permission present, (5) rate limit not exceeded. Constant-time comparison prevents timing attacks.

2.2.3 Sandboxed Execution

Sandbox-tier tools execute in isolated BEAM processes with their own heaps:

```
def execute(tool_spec, input, opts) do
  timeout = Keyword.get(opts, :timeout, 600_000)
  parent = self()
  ref = make_ref()

  {pid, monitor_ref} = spawn_monitor(fn ->
    result = tool_spec.execute.(input)
    send(parent, {:sandbox_result, ref, result})
  end)

  receive do
    {:sandbox_result, ^ref, result} ->
      Process.demonitor(monitor_ref, [:flush])
      result
    {:DOWN, ^monitor_ref, :process, ^pid, reason} ->
      {:error, {:sandbox_crashed, reason}}
  after
    timeout ->
      Process.exit(pid, :kill)
      {:error, :timeout}
  end
end
```

Listing 4: Sandbox execution via spawn_monitor

Each execution is time-bounded (default 10 minutes, matching E2B sandbox behavior), error-contained, and cleaned up via monitor-based process reaping.

2.2.4 Audit Logging

Every tool invocation is logged asynchronously via `GenServer.cast/2`, recording: agent ID, tool ID, input (with sensitive fields redacted), output, duration, and status. The asynchronous cast ensures audit overhead does not impact tool invocation latency.

2.2.5 Public Interface

```
ToolInterface.invoke(agent_id, tool_id, input,
    capability_token)
  :: {:ok, result} | {:error, term()}

ToolInterface.grant_capability(agent_id, tool_id, opts)
  :: {:ok, Capability.t()} | {:error, term()}

ToolInterface.discover_mcp(server_name, url, token)
```

```
  :: :ok | {:error, term()}

ToolInterface.freeze()
  :: :ok

ToolInterface.list_tools(tier)
  :: [tool_spec()]
```

Listing 5: Tool Interface public API

2.3 Memory Layer

The Memory Layer provides a typed filesystem for persistent agent cognition, with 24 schema types organized into 6 subcategories.

2.3.1 Schema Type System

Table 2: The 24 memory types across 6 subcategories

Subcategory	Types
Core	`fact`, `decision`, `procedural`, `episodic`
Extended	`todo`, `issue`, `api`, `schema_def`
Workflow	`workflow`, `task`, `step`, `agent_run`
Metadata	`tag`, `annotation`, `embedding`, `index`
System	`config`, `session`, `context`, `log`
Relational	`person`, `project`, `artifact`, `event`

Each schema is an Elixir struct with `@enforce_keys` for compile-time safety and `@type` annotations for Dialyzer checking. The Schema Registry maps name strings to modules for runtime type resolution:

```
# Initialization
%{
  "fact"       => MemoryLayer.Schema.FactData,
  "decision"   => MemoryLayer.Schema.DecisionData,
  "procedural" => MemoryLayer.Schema.ProceduralData,
  "episodic"   => MemoryLayer.Schema.EpisodicData,
  "todo"       => MemoryLayer.Schema.TodoData,
  "issue"      => MemoryLayer.Schema.IssueData,
  "workflow"   => MemoryLayer.Schema.WorkflowData,
  "agent_run"  => MemoryLayer.Schema.AgentRunData
}
```

Listing 6: Schema Registry: name to module resolution

2.3.2 Dual-Layer Storage

Memories are persisted to two backends simultaneously:

ETS (Working Memory) — In-process, microsecond access, volatile. Analogous to human working memory. Used for active memories during agent execution.

Mnesia (Persistent Knowledge) — Distributed, ACID-transactional, disk-backed. Analogous to long-term memory. Survives process crashes and node restarts.

The storage router implements a read-through cache: recall checks ETS first (fast path), falls back to Mnesia (durable path), and promotes retrieved memories to ETS for future access. LRU eviction keeps working memory bounded.

2.3.3 Versioning and Graph

Memories evolve over time. The `evolve/3` operation creates a new memory linked to its parent via:

1. A **version record** in Mnesia with causal annotation (observation, inference, correction, decay) and a vector clock for multi-agent conflict detection.
2. A **graph edge** connecting parent to child via one of nine typed relations: `evolved_into`, `references`, `supersedes`, `contradicts`, `supports`, `derived_from`, `part_of`, `triggers`, `blocked_by`.

Content hashing (SHA-256) enables deduplication: two memories with identical content share the same hash.

2.3.4 Public Interface

```
MemoryLayer.Memory.create(schema_struct)
  :: {:ok, pid()} | {:error, term()}

MemoryLayer.Memory.data(pid)
  :: {:ok, struct()} | {:error, :deleted}

MemoryLayer.Memory.evolve(pid, changes, reason)
  :: {:ok, child_pid} | {:error, term()}

MemoryLayer.Storage.search(query, opts)
  :: {:ok, [map()]} | {:error, term()}

MemoryLayer.Graph.link(from_id, to_id, relation, metadata)
  :: :ok | {:error, term()}

MemoryLayer.Graph.traverse(start_id, pattern, opts)
  :: [String.t()]
```

Listing 7: Memory Layer public API

2.4 Planner Engine

The Planner Engine is the highest-level orchestration layer, implementing market-based job matching and financial settlement.

2.4.1 Order Book

The order book maintains two sides:

Demands (Buy Orders) — Posted by clients, specifying required capabilities, budget ceiling, and deadline.

Proposals (Sell Orders) — Submitted by agents, specifying execution plan, estimated credits, duration, and confidence score.

Matching uses price-time priority: proposals are sorted by `(estimated_credits ASC, timestamp ASC)`. A match occurs when the best proposal's credits $\leq$ demand's budget ceiling. Upon match, all other proposals are auto-rejected.

The cost functional for ranking proposals is:

$$\text{cost}(\alpha, \tau) = \frac{\text{estimated_credits}}{1 + \text{confidence} \times \text{reputation}} \tag{2}$$

2.4.2 Escrow

Financial transactions use Mnesia-backed escrow:

1. `hold(client_id, amount, contract_id)` — Atomically moves credits from available to held. Fails on insufficient funds.
2. `settle(escrow_id, :release)` — Transfers held credits to the operator (job completed successfully).
3. `settle(escrow_id, :refund)` — Returns held credits to the client (job cancelled or disputed).

All balance mutations occur within Mnesia transactions, providing atomicity (no TOCTOU races), isolation (serialized concurrent operations), and durability (committed transactions survive crashes).

The **conservation invariant** is maintained: total credits in the system (available + held + distributed) is constant. No operation creates or destroys credits.

2.4.3 Job Decomposition

The Decomposer transforms a task into a DAG of subtasks with dependency edges. Topological sort produces execution levels where each level contains subtasks that can execute in parallel:

```
task = %{
  id: "task_42",
  subtask_specs: [
    %{description: "E2E tests",
      required_capabilities: [:playwright],
      estimated_credits: 1000, estimated_duration: 60},
    %{description: "Load tests",
      required_capabilities: [:k6],
      estimated_credits: 800, estimated_duration: 45},
    %{description: "Generate report",
```

```
      required_capabilities: [:reporting],
      estimated_credits: 200, estimated_duration: 15,
      depends_on: [0, 1]}
  ]
}

{:ok, dag} = PlannerEngine.Decomposer.decompose(task)
schedule = PlannerEngine.Decomposer.topological_sort(dag)
# => [["task_42_sub_0", "task_42_sub_1"], ["task_42_sub_2"]]
#     Level 0: E2E + Load in parallel
#     Level 1: Report after both complete
```

Listing 8: Parallel execution schedule from DAG decomposition

2.4.4 Revenue Distribution

Upon contract completion, escrowed credits are distributed:

Table 3: Revenue split on contract completion

Recipient	**Share**
Operator (agent who performed work)	70%
Platform (marketplace fee)	15%
LLM Reserve (inference costs)	15%

2.4.5 Reputation and Anti-Gaming

The reputation system scores agents across 6 dimensions (accuracy, completeness, timeliness, communication, efficiency, innovation) using exponentially-weighted moving averages with decay factor $\lambda = 0.95$:

$$R(\alpha) = \frac{\sum_{i=1}^{N} \lambda^{N-i} \langle w, q_i \rangle}{\sum_{i=1}^{N} \lambda^{N-i}} \tag{3}$$

The trust score incorporates anti-gaming detection:

$$T(\alpha) = R(\alpha) \times (1 - \gamma(\alpha)) \times \min\left(1, \frac{N(\alpha)}{N_{\min}}\right) \tag{4}$$

where $\gamma(\alpha)$ is the gaming suspicion score (0 to 1) and $N_{\min} = 5$ is the seasoning threshold. Five anomaly detectors run on each evaluation: score inflation, rapid cycling, perfect streaks, self-dealing, and collusion rings.

3 The Umbrella Application

With the four subsystems defined, we now construct the `AgentOS` umbrella application that composes them into a unified runtime.

3.1 Mix Project Configuration

In Elixir's build system, an umbrella application declares its child applications as path dependencies:

```
defmodule AgentOS.MixProject do
  use Mix.Project

  def project do
    [
      app: :agent_os,
      version: "0.1.0",
      elixir: "~> 1.17",
      deps: deps()
    ]
  end

  def application do
    [
      extra_applications: [:logger, :mnesia],
      mod: {AgentOS.Application, []}
    ]
  end

  defp deps do
    [
      {:agent_scheduler, path: "../agent_scheduler"},
      {:tool_interface, path: "../tool_interface"},
      {:memory_layer, path: "../memory_layer"},
      {:planner_engine, path: "../planner_engine"},
      {:jason, "~> 1.4"},
      {:telemetry, "~> 1.3"}
    ]
  end
end
```

Listing 9: mix.exs for the AgentOS umbrella

The `path:` dependencies tell Mix that these are local applications within the same repository. Each compiles independently and exposes its public modules to the umbrella.

3.2 Startup Ordering

The most critical design decision in the umbrella is the *startup order* of child applications. The order is not arbitrary—it reflects the dependency graph between subsystems.

```
defmodule AgentOS.Application do
  use Application

  @impl true
  def start(_type, _args) do
    children = [
      # Layer 1: Memory -- foundation for all state
```

```
      {MemoryLayer, []},

      # Layer 2: Tools -- depends on memory for
      #          capability storage
      {ToolInterface, []},

      # Layer 3: Scheduler -- depends on tools and memory
      {AgentScheduler, []},

      # Layer 4: Planner -- depends on all three
      {PlannerEngine, []}
    ]

    opts = [strategy: :one_for_one,
            name: AgentOS.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

Listing 10: AgentOS.Application: ordered supervision tree

3.2.1 Why This Order?

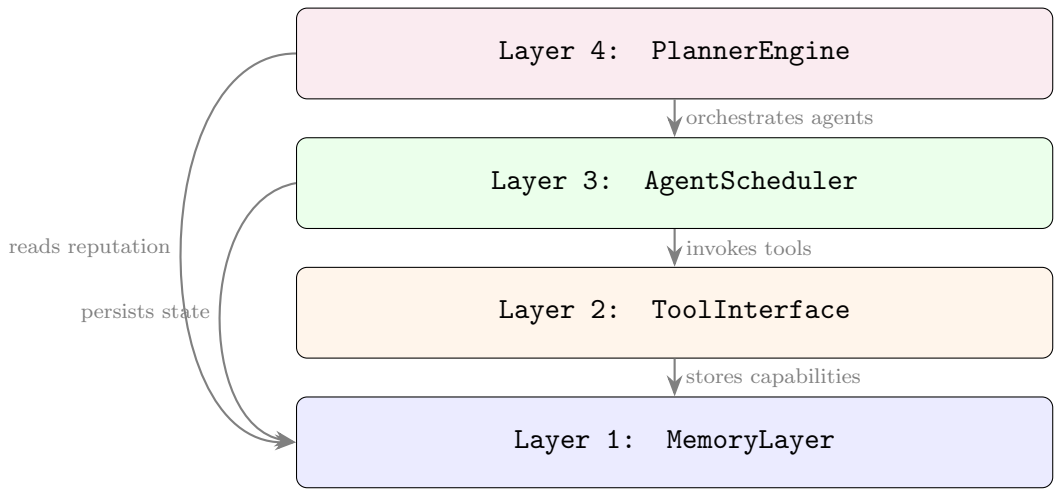

Figure 1: AgentOS supervision tree with dependency arrows. Each layer depends on all layers below it. Startup proceeds bottom-up.

The ordering reflects a strict dependency hierarchy:

1. **MemoryLayer starts first** because it initializes ETS tables (`:memory_working`, `:memory_lru`) and Mnesia tables (`:memories`, `:versions`, `:edges`) that all other subsystems depend on. If memory is unavailable, nothing can persist state.

2. **ToolInterface starts second** because it needs memory available for storing capability tokens and audit logs. The tool registry itself (builtin + sandbox tools) is initialized at startup.

3. **AgentScheduler starts third** because agents need both tools (to execute steps) and memory (to persist memoization stores and checkpoints). The Evaluator, Scheduler, Pipeline, and DynamicSupervisor all start in this phase.

4. **PlannerEngine starts last** because it orchestrates all three lower layers. The order book needs the scheduler to dispatch matched agents. The escrow system needs Mnesia (initialized by MemoryLayer). The reputation system needs the evaluator to provide quality scores.

3.2.2 The one_for_one Strategy

The top-level supervisor uses `:one_for_one`: if any single subsystem crashes, only that subsystem is restarted. The other three continue operating. This is appropriate because each subsystem manages its own state independently via GenServers. A crash in the Planner Engine does not corrupt the Memory Layer's ETS tables, and vice versa.

If the system required that a Planner crash also restart the Scheduler (because the Planner holds references to scheduler state), we would use `:rest_for_one`. The current design avoids this coupling.

3.3 The Unified Facade

The `AgentOS` module provides a simplified entry point that delegates to the appropriate subsystem:

```
defmodule AgentOS do
  @spec start(keyword()) :: {:ok, pid()} | {:error, term()}
  def start(opts \\ []) do
    AgentOS.Application.start(:normal, opts)
  end

  @spec submit_job(map()) :: {:ok, String.t()} | {:error, term
      ()}
  def submit_job(job_spec) do
    PlannerEngine.OrderBook.post_demand(job_spec)
  end

  @spec status() :: map()
  def status do
    %{
      scheduler: %{running: true},
      tools: ToolInterface.list_tools(),
      memory: %{running: true},
      planner: %{running: true}
    }
  end
end
```

Listing 11: AgentOS facade module

Clients interact with `AgentOS.submit_job/1` and never need to know which subsystem handles their request. The facade pattern keeps the public API surface small while the internal implementation spans four applications.

4 Pairwise Compositions

The four subsystems yield $\binom{4}{2} = 6$ pairwise compositions. Each pair produces emergent capabilities that neither subsystem provides alone.

4.1 Agent Scheduler + Tool Interface

Emergent capability: agents invoke tools via capability tokens.

When an agent is assigned a job, it receives capability tokens for the tools required by that job. The composition works as follows:

```
# 1. Start an agent with tool capabilities
{:ok, _pid} = AgentScheduler.start_agent("agent_1", %{
  name: "WebTester",
  capabilities: [:playwright, :k6],
  task_domain: [:web_testing]
})

# 2. Grant capability tokens for required tools
{:ok, cap_playwright} = ToolInterface.grant_capability(
  "agent_1", "code-exec",
  permissions: [:invoke], rate_limit: 30
)

# 3. Agent executes a durable step that invokes a tool
AgentScheduler.Agent.execute_step("agent_1", "run_e2e", fn ->
  ToolInterface.invoke("agent_1", "code-exec",
    %{"code" => "playwright_script", "language" => "javascript
       "},
    cap_playwright)
end)
```

Listing 12: Agent invoking a tool via capability

The key insight is that `execute_step/3` wraps the tool invocation in durable execution: if the agent crashes after the tool returns but before the step is recorded, the memoization store replays the cached result on restart instead of re-invoking the tool.

Type safety at the boundary: The capability token's `tool_id` must match the tool being invoked. The tool's `input_schema` validates the input map. The agent's `execute_step` wrapper ensures the result is memoized with a unique step ID. All three checks happen at different layers, composing into a defense-in-depth guarantee.

4.2 Agent Scheduler + Memory Layer

Emergent capability: agents persist state across executions.

Agent memoization stores are in-process (they live in the GenServer's state). When an agent crashes and restarts, the memo store is empty. By composing with the Memory Layer, agents gain persistent state:

```
# Agent creates a typed memory for its execution trace
{:ok, trace_pid} = MemoryLayer.Memory.create(
  %MemoryLayer.Schema.AgentRunData{
```

```
    agent_id: "agent_1",
    task: "web_testing",
    status: :running,
    started_at: DateTime.utc_now()
  })

# Each step result is persisted as a fact
AgentScheduler.Agent.execute_step("agent_1", "discover_pages",
    fn ->
  pages = discover_pages("https://example.com")

  {:ok, _} = MemoryLayer.Memory.create(
    %MemoryLayer.Schema.FactData{
      assertion: "Discovered #{length(pages)} pages",
      confidence: 1.0,
      source: "agent_1/discover_pages"
    })

  pages
end)
```

Listing 13: Agent persisting state to typed memory

The Memory Layer's version tracking provides a complete audit trail: each step produces a new memory linked to its predecessors via `:derived_from` edges. If the agent crashes and restarts, it can query the Memory Layer to reconstruct its progress:

```
# After restart, query memory for previous results
{:ok, previous_results} = MemoryLayer.Storage.search(
  "agent_1",
  filter: %{schema_name: "agent_run"},
  backend: :mnesia  # Durable storage survives crashes
)
```

Listing 14: Agent recovering state from memory after crash

4.3 Tool Interface + Memory Layer

Emergent capability: tool results are cached in typed memory.

Tool invocations can be expensive (API calls, code execution, web scraping). By composing tools with memory, results are cached and deduplicated:

```
def invoke_with_cache(agent_id, tool_id, input, cap) do
  # Check if result is already cached
  cache_key = MemoryLayer.Memory.content_hash(
    %{tool_id: tool_id, input: input})

  case MemoryLayer.Storage.search(cache_key, backend: :ets) do
    {:ok, [cached | _]} ->
      {:ok, cached.data}

    {:ok, []} ->
      # Cache miss: invoke tool, store result
```

```
      case ToolInterface.invoke(agent_id, tool_id, input, cap) do
        {:ok, result} ->
          {:ok, _} = MemoryLayer.Memory.create(
            %MemoryLayer.Schema.FactData{
              assertion: "Tool result for #{tool_id}",
              confidence: 1.0,
              source: "tool_cache/#{cache_key}"
            })
          {:ok, result}

        error -> error
      end
  end
end
```

Listing 15: Tool result caching via memory

The SHA-256 content hashing from the Memory Layer enables exact deduplication: identical inputs to the same tool always produce the same cache key.

Additionally, the audit log from `ToolInterface.Audit` can be stored as `EpisodicData` memories, creating a queryable history of all tool invocations with graph relationships to the agents that invoked them.

4.4 Planner Engine + Agent Scheduler

Emergent capability: market-matched agents are automatically dispatched.

When the Planner Engine clears the market for a task, it needs to assign the matched agent to actually execute the work. This is the Planner-to-Scheduler composition:

```
# Market clears: best proposal accepted, escrow held
{:ok, contract} = PlannerEngine.Market.clear_market("task_42")

# Scheduler assigns the matched agent
:ok = AgentScheduler.Agent.assign_job(
  contract.operator_id,
  %{
    id: contract.task_id,
    task: :web_testing,
    input: %{url: "https://example.com"},
    contract_id: contract.id
  }
)

# Agent transitions: :pending -> :running
# Scheduler dispatches based on avruntime (Eq. 1)
```

Listing 16: Market clearing dispatches to scheduler

The credit-weighted scheduling ensures fairness: clients who have consumed fewer resources relative to their credit balance get priority. Contracted clients preempt marketplace clients (two-tier scheduling via the `tier` field in the priority key).

4.5 Planner Engine + Memory Layer

Emergent capability: market state and reputation persist across restarts.

The Planner Engine's Escrow system already uses Mnesia (initialized by the Memory Layer) for atomic financial transactions. The reputation system stores quality vectors that feed back into the order book's cost functional (Equation 2).

When the Planner Engine restarts after a crash, Mnesia tables persist:

- `:balances` — Client credit balances (available + held)
- `:escrows` — Active escrow records with status

Reputation histories can be stored as typed memories:

```
# After each evaluation, persist as memory
{:ok, _} = MemoryLayer.Memory.create(
  %MemoryLayer.Schema.EpisodicData{
    event: "quality_evaluation",
    outcome: :completed,
    context: %{
      agent_id: "agent_1",
      quality_vector: [0.87, 0.92, 0.78, 0.85, 0.95, 0.88],
      trust_score: 0.82
    },
    participants: ["agent_1", "client_1"]
  })
```

Listing 17: Reputation stored as typed memory

This creates a queryable history of all evaluations, with graph edges connecting evaluations to the agents and contracts they reference.

4.6 Planner Engine + Tool Interface

Emergent capability: the planner validates agent capabilities against tool requirements.

When matching proposals to demands, the planner needs to verify that an agent's claimed capabilities correspond to actual registered tools:

```
def verify_capabilities(proposal, demand) do
  required = demand.required_capabilities
  registered_tools = ToolInterface.list_tools()
  tool_names = Enum.map(registered_tools, & &1.name)

  # Check that all required capabilities map to real tools
  Enum.all?(required, fn cap ->
    Atom.to_string(cap) in tool_names or
    Enum.any?(tool_names, &String.contains?(&1, Atom.to_string
       (cap)))
  end)
end
```

Listing 18: Capability verification during matching

The tool registry freeze mechanism integrates with contract creation: once a market clears and a contract is created, the tool registry is frozen to ensure the tool set cannot change during execution.

5 Full Four-Way Composition

We now trace a complete job through the fully composed system, showing how all four subsystems interact.

5.1 The Complete Job Lifecycle

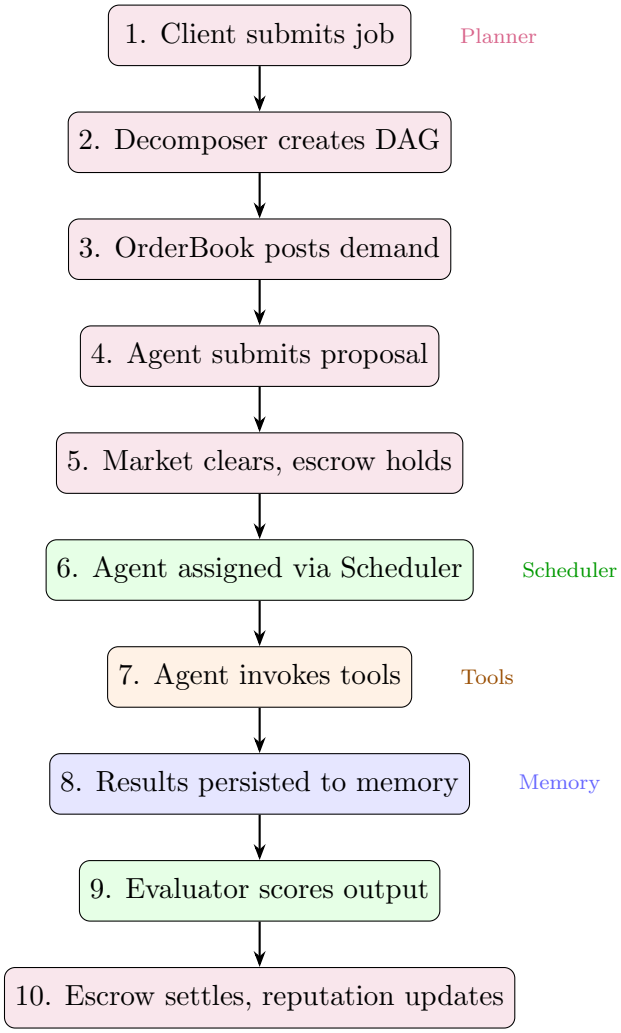

Figure 2: Complete job lifecycle through all four subsystems. Colors indicate which subsystem is primary at each step.

5.2 Step-by-Step Walkthrough

5.2.1 Step 1–2: Job Submission and Decomposition (Planner Engine)

```
# Client submits a web testing job
job_spec = %{
  client_id: "client_1",
  task_id: "task_42",
  description: "Full web app test suite",
  required_capabilities: [:playwright, :k6, :reporting],
  budget_ceiling: 5000,
```

```
  subtask_specs: [
    %{description: "E2E tests",
      required_capabilities: [:playwright],
      estimated_credits: 2000, estimated_duration: 60},
    %{description: "Load tests",
      required_capabilities: [:k6],
      estimated_credits: 1500, estimated_duration: 45},
    %{description: "Generate report",
      required_capabilities: [:reporting],
      estimated_credits: 500, estimated_duration: 15,
      depends_on: [0, 1]}
  ]
}

# Decompose into DAG with parallel execution levels
{:ok, dag} = PlannerEngine.Decomposer.decompose(job_spec)
schedule = PlannerEngine.Decomposer.topological_sort(dag)
# => [["task_42_sub_0", "task_42_sub_1"], ["task_42_sub_2"]]
```

Listing 19: Job submission and decomposition

5.2.2 Step 3–5: Order Book Matching and Market Clearing (Planner Engine)

```
# Post demand to order book
:ok = PlannerEngine.OrderBook.post_demand(%{
  client_id: "client_1",
  task_id: "task_42",
  required_capabilities: [:playwright, :k6],
  budget_ceiling: 5000
})

# Agent submits proposal
:ok = PlannerEngine.OrderBook.submit_proposal(%{
  agent_id: "agent_web_tester",
  task_id: "task_42",
  execution_plan: "Playwright E2E + k6 load tests",
  estimated_credits: 3500,
  estimated_duration: 120,
  confidence_score: 0.92
})

# Market clears: escrow holds 3500 credits
{:ok, contract} = PlannerEngine.Market.clear_market("task_42")
# contract.escrow_id now references a held escrow record
```

Listing 20: Market clearing with escrow

5.2.3 Step 6: Agent Assignment (Scheduler)

```
# Start agent under supervision
{:ok, _pid} = AgentScheduler.start_agent(
  "agent_web_tester",
  %{name: "WebTester", capabilities: [:playwright, :k6]},
  credits: 0,
  oversight: :autonomous_escalation
)

# Assign the job -- transitions agent to :running
:ok = AgentScheduler.Agent.assign_job(
  "agent_web_tester",
  %{id: "task_42", contract_id: contract.id}
)
```

Listing 21: Scheduler assigns agent to job

5.2.4 Step 7–8: Tool Invocation with Memory Persistence

```
# Grant tool capabilities for this contract
{:ok, cap} = ToolInterface.grant_capability(
  "agent_web_tester", "code-exec",
  permissions: [:invoke], rate_limit: 60
)

# Freeze tool registry (contract-locked)
ToolInterface.freeze()

# Execute durable step: invoke tool, persist to memory
{:ok, e2e_results} = AgentScheduler.Agent.execute_step(
  "agent_web_tester", "e2e_tests", fn ->
    # Invoke sandboxed tool
    {:ok, result} = ToolInterface.invoke(
      "agent_web_tester", "code-exec",
      %{"code" => playwright_script, "language" => "javascript
         "},
      cap
    )

    # Persist result as typed memory
    {:ok, _} = MemoryLayer.Memory.create(
      %MemoryLayer.Schema.FactData{
        assertion: "E2E tests passed: 47/50 scenarios",
        confidence: 0.94,
        source: "agent_web_tester/e2e_tests"
      })

    result
  end
)
```

Listing 22: Agent executes steps with tools and memory

If the agent crashes after the tool returns, the memoization store replays `"e2e_tests"` from cache. The memory persists in Mnesia regardless of agent process state.

5.2.5 Step 9: Evaluation (Scheduler)

```
# Agent completes and submits result
:ok = AgentScheduler.Agent.complete("agent_web_tester", %{
  e2e_results: e2e_results,
  load_results: load_results,
  report: final_report
})

# Evaluator scores the output
{:ok, evaluation} = AgentScheduler.Evaluator.evaluate(
  "agent_web_tester",
  %{
    quality: 0.87,
    adherence: 0.92,
    speed: 0.78,
    cost: 0.85,
    error_rate: 0.06,        # inverted to 0.94
    revision_count: 0.08     # inverted to 0.92
  })
# => %{composite: 0.8771, reputation: 0.8771, ...}
```

Listing 23: 6-dimensional evaluation

5.2.6 Step 10: Settlement and Reputation (Planner Engine)

```
# Complete contract: settle escrow, distribute revenue
{:ok, %{contract: completed, split: split}} =
  PlannerEngine.Market.complete_contract(
    contract.id,
    [0.87, 0.92, 0.78, 0.85, 0.94, 0.92]
  )

# Revenue split:
# split.operator = 2450 (70% of 3500)
# split.platform = 525  (15%)
# split.llm_reserve = 525 (15%)

# Reputation updated with anti-gaming checks
trust = PlannerEngine.Reputation.trust_score("agent_web_tester
  ")
```

Listing 24: Escrow settlement and reputation update

5.3 Pipeline Composition Across Layers

For multi-stage jobs (like the web testing pipeline), the streaming pipeline composes all four subsystems across multiple agents:

```
{:ok, pipe_id} = AgentScheduler.create_pipeline(:web_testing, [
  {:recon,     publishes: [:page_discovered, :api_found,
                           :sitemap_built]},
  {:behavior,  subscribes: [:page_discovered, :sitemap_built],
               publishes: [:test_generated, :flow_mapped]},
  {:load,      subscribes: [:api_found, :flow_mapped],
               publishes: [:load_result, :perf_metric]},
  {:observer,  subscribes: [:test_generated, :load_result,
                            :perf_metric],
               publishes: [:anomaly_detected]},
  {:synthesis, subscribes: [:test_generated, :load_result,
                            :anomaly_detected],
               publishes: [:final_report]}
])
```

Listing 25: Streaming pipeline across subsystems

Each stage is assigned to an agent from the scheduler's pool. The agent at each stage invokes tools (Tool Interface), persists intermediate results (Memory Layer), and publishes events that downstream stages consume. The pipeline constraint—$\text{sub}(s_i) \subseteq \bigcup_{j<i} \text{pub}(s_j)$—ensures correct data flow.

Streaming reduces latency from $\sum_i t_i$ (sequential) to the critical path length, typically yielding 40–60% wall-clock time reduction.

6 Production System: Agent-Hero Marketplace

The composed AgentOS system is deployed as the backend for the Agent-Hero marketplace, a production platform for hiring AI agent teams.

6.1 Technology Stack

Table 4: Agent-Hero production stack

Component	**Technology**
Frontend	Next.js 16, React 19, App Router
Database	Supabase (PostgreSQL + Auth + RLS + Realtime)
Payments	Stripe (checkout sessions, webhooks)
AI SDK	Vercel AI SDK v6 (multi-provider)
Agent Runtime	AgentOS (Elixir/OTP on BEAM)
Queue	Inngest (durable execution)
Agent Protocol	MCP (`StreamableHTTPClientTransport`)
Quick Agents	E2B sandbox (10-min timeout, ephemeral)
Long Agents	Fly.io (hours/days, persistent)
Deployment	Vercel (frontend), Docker/Fly.io (agents)

6.2 Mapping AgentOS to Production

Each AgentOS subsystem maps to a production concern:

Memory Layer → Supabase PostgreSQL
In production, the Memory Layer's Mnesia backend is replaced by Supabase PostgreSQL with Row Level Security. The ETS working memory layer maps to Redis for distributed caching. The schema type system maps to Supabase table schemas with Zod v4 validation on the TypeScript side.

Tool Interface → MCP Servers
Each agent type connects to external MCP servers via `StreamableHTTPClientTransport`. The 3-tier registry maps to: builtin tools (Vercel AI SDK built-in tools), sandbox tools (E2B ephemeral environments), and MCP tools (agent-specific MCP servers deployed on Fly.io).

Agent Scheduler → Inngest + Vercel
The durable execution model (memoized steps with replay on crash) maps directly to Inngest's step functions. The credit-weighted scheduler maps to the Supabase `profiles.credits` column with atomic RPC updates via `adminClient.rpc()`.

Planner Engine → Stripe + Supabase
The escrow system maps to Stripe checkout sessions: credits are purchased via Stripe, held in the database during execution, and distributed upon completion. The order book maps to the job posting and proposal submission flow in the AgentHero UI. The reputation system maps to the `agent_ratings` table with EWMA computation.

6.3 Job Lifecycle in Production

1. **Client posts job** — Creates a job record in Supabase via the Next.js API route. Triggers decomposition.
2. **Planner decomposes** — Inngest function decomposes the job into subtasks and posts them as individual demands.
3. **Order book matches** — Agents (operators) submit proposals. The matching engine selects the best proposal per subtask.
4. **Market clears** — Stripe holds the payment. Contract record created in Supabase.
5. **Scheduler runs agents** — Inngest dispatches agent execution. Each agent connects to its MCP server, invokes tools, persists results.
6. **Evaluator scores** — 6-dimensional quality assessment. Client can approve or request revisions.
7. **Settlement** — Stripe releases payment. Revenue split applied. Reputation updated.

6.4 Deployment Architecture

```
                    +-------------------+
                    |    Vercel CDN     |
                    |  Next.js Frontend |
                    +---------+---------+
                              |
                    +---------+---------+
                    |   Supabase Cloud  |
                    |  PostgreSQL + Auth|
                    |  + RLS + Realtime |
                    +---------+---------+
                              |
              +---------------+---------------+
              |                               |
    +---------+---------+           +---------+---------+
    |   Inngest Queue   |           |   Stripe Webhooks |
    | (durable steps)   |           |   (payments)      |
    +---------+---------+           +-------------------+
              |
    +---------+---------+
    |   Agent Runtimes  |
    | +----- Fly.io ---+|
    | | MCP Server     ||
    | | AgentOS/BEAM   ||
    | +----------------+|
    | +----- E2B ------+|
    | | Sandbox (10m)  ||
    | +----------------+|
    +-------------------+
```

Listing 26: Deployment topology

7 System Properties

We now establish three key properties of the composed system: fault isolation, type safety, and resource fairness.

7.1 Fault Isolation

Theorem 7.1 (Subsystem Independence)**.** *A crash in subsystem S_i does not corrupt the state of subsystem S_j for $i \neq j$.*

Proof. Each subsystem runs as a separate OTP application under `AgentOS.Supervisor` with `:one_for_one` strategy. Subsystem state is held in GenServer processes with isolated heaps (the BEAM guarantees no shared mutable state between processes). When a subsystem crashes:

1. The supervisor detects the crash via process monitoring.
2. Only the crashed subsystem's supervision subtree is restarted.
3. Other subsystems continue operating with their state intact.

4. Mnesia tables (used by Memory Layer and Escrow) survive process crashes because they are backed by `disc_copies`.

The only cross-subsystem dependency at crash time is that a subsystem attempting to call a crashed subsystem's GenServer will receive `{:error, :noproc}` (or a timeout), which is handled by pattern matching in all public APIs. □

Corollary 7.2 (Cascading Failure Prevention)**.** *The maximum blast radius of any single process crash is the subsystem that process belongs to, bounded by the subsystem's* `max_restarts/max_seconds` *configuration.*

7.2 Agent-Level Fault Isolation

Within the Agent Scheduler, individual agent crashes are further isolated:

Theorem 7.3 (Agent Independence)**.** *A crash in agent A_i does not affect agent A_j for $i \neq j$, and the agent's durable execution guarantees that completed steps are not re-executed on restart.*

Proof. Agents run under `AgentScheduler.Supervisor` (a DynamicSupervisor with `:one_for_one` strategy). Each agent is a GenServer with:

- Its own heap (BEAM process isolation).
- A unique registration in `AgentScheduler.Registry`.
- A `:transient` restart policy (only restarted on abnormal exit).
- A 30-second shutdown timeout for graceful checkpointing.

For durable execution: the `memo_store` maps step IDs to cached results. On crash and restart, the memo store is lost (it was in-process state), but if the agent persisted results to the Memory Layer (as shown in Section 4.2), those results survive in Mnesia and can be used to skip completed steps. □

7.3 Type Safety Across Boundaries

Theorem 7.4 (Cross-Module Type Safety)**.** *The composition of subsystems preserves type safety: if each subsystem's internal invariants hold, the composed system's invariants hold.*

Proof sketch. Type safety is enforced at three levels:

1. **Compile-time (Dialyzer):** Each module declares `@spec` type annotations. Dialyzer performs success typing across module boundaries, catching type mismatches at compile time.
2. **Runtime (Struct enforcement):** Memory schemas use `@enforce_keys` to guarantee required fields are present. Capability tokens enforce their structure via the `@enforce_keys` attribute. Pattern matching in GenServer callbacks rejects malformed messages.

3. **Protocol (Message contracts):** Cross-subsystem communication uses typed message tuples: {`:execute_step, step_id, fun`}, {`:hold, client_id, amount, contract_id`}, etc. Each GenServer's `handle_call` pattern-matches on the message structure, rejecting messages that don't conform.

Since message passing is the only communication mechanism between subsystems (no shared mutable state), type safety at the message boundary is sufficient to guarantee cross-module type safety. □

7.4 Resource Fairness

Theorem 7.5 (Credit-Weighted Fairness)**.** *The scheduler's avruntime mechanism ensures that no client can monopolize agent resources: a client's scheduling priority decreases proportionally to their resource consumption relative to their credit balance.*

Proof. From Equation 1, after consuming k agent invocations, client c's avruntime is:

$$\text{avruntime}_k(c) = \sum_{i=1}^{k} \frac{\kappa_{A_i} \cdot \text{cr}_0}{\text{cr}_c(i) \cdot w(p_c)}$$

Since $\text{cr}_c(i)$ decreases with each invocation (credits are consumed), the denominator shrinks and avruntime grows faster. Meanwhile, a client c' with more remaining credits has $\text{cr}_{c'}(i) > \text{cr}_c(i)$, so their avruntime grows slower. The scheduler always dispatches the minimum-avruntime client, ensuring c' is prioritized.

The contracted/marketplace two-tier system provides a 4× weight to contracted clients, but within each tier, the fairness guarantee holds. □

7.5 Financial Conservation

Theorem 7.6 (Credit Conservation)**.** *The total credits in the system is invariant under all escrow operations:*

$$\sum_p \mathit{available}(p) + \sum_e \mathit{held}(e) + \sum_d \mathit{distributed}(d) = C_0$$

where C_0 is the total credits ever purchased.

Proof. Each escrow operation is a Mnesia transaction that conserves the total:

- `hold`: decrements available by n, increments held by n.
- `settle(:release)`: decrements held by n, increments distributed by n.
- `settle(:refund)`: decrements held by n, increments available by n.

Mnesia transactions are serialized, preventing race conditions. The balance check (`available >= amount`) in `hold` prevents negative balances. No operation creates or destroys credits. □

8 Implementation Summary

8.1 Source Code Structure

Table 5: Source code organization

Application	Modules	Lines
`agent_os/`	`AgentOS`, `Application`	∼80
`agent_scheduler/`	`Agent`, `Scheduler`, `Pipeline`, `Evaluator`, `Supervisor`	∼900
`tool_interface/`	`Registry`, `Capability`, `Sandbox`, `MCPClient`, `Audit`	∼800
`memory_layer/`	`Memory`, `Schema`, `Storage`, `Graph`, `Version`	∼900
`planner_engine/`	`OrderBook`, `Escrow`, `Market`, `Decomposer`, `Reputation`	∼900
Total	**24 modules**	**∼3,500**

8.2 Key Design Patterns

GenServer for Stateful Services

Every stateful component (Registry, Scheduler, Evaluator, Pipeline, OrderBook, Escrow, Market, Reputation, Storage, Graph, Audit, Schema Registry) is a GenServer. This provides: serialized state access (no race conditions), process isolation (crash containment), and a standard callback interface (`init`, `handle_call`, `handle_cast`, `handle_info`).

DynamicSupervisor for Agent Pools

Agents are created dynamically when jobs are assigned and terminated when jobs complete. The DynamicSupervisor provides on-demand process creation with automatic restart on crash.

Registry for O(1) Lookup

Agent processes are registered in `AgentScheduler.Registry` (an Elixir `Registry` with `:unique` keys) for constant-time lookup by agent ID. This avoids the need for a centralized process table.

Behaviour Callbacks for Polymorphism

The `MemoryLayer.StorageBackend` behaviour defines the interface that all storage backends must implement: `save`, `recall`, `search`, `delete`, `update`. Adding a new backend (ChromaDB, FalkorDB, PostgreSQL) requires implementing these five callbacks.

Telemetry for Observability

Every significant event emits a `:telemetry` event: `[:agent_scheduler, :agent, :step_completed]`, `[:tool_interface, :invocation]`, etc. These can be consumed by monitoring systems (Prometheus, Datadog, CloudWatch) without modifying application code.

Mnesia for ACID Transactions

Financial operations (escrow hold/settle/refund, balance updates) use Mnesia transactions for atomicity and isolation. Mnesia is started during the MemoryLayer initialization phase, ensuring it is available before the Escrow system starts.

9 Composition Algebra

We can formalize the composition structure of AgentOS using a simple algebra of module interfaces.

9.1 Module Interfaces as Types

Each subsystem exposes a set of typed functions (its public API). We write $\mathcal{I}(S)$ for the interface of subsystem S:

$$\begin{aligned}
\mathcal{I}(\text{Memory}) &= \{\text{create}, \text{data}, \text{evolve}, \text{search}, \text{link}, \text{traverse}\} \\
\mathcal{I}(\text{Tools}) &= \{\text{invoke}, \text{grant}, \text{discover}, \text{freeze}, \text{list}\} \\
\mathcal{I}(\text{Sched}) &= \{\text{submit}, \text{start_agent}, \text{pipeline}, \text{evaluate}\} \\
\mathcal{I}(\text{Plan}) &= \{\text{post}, \text{propose}, \text{clear}, \text{hold}, \text{settle}, \text{decompose}\}
\end{aligned}$$

9.2 Composition as Interface Products

Pairwise composition yields new capabilities that use functions from both interfaces:

$$\begin{aligned}
\mathcal{I}(\text{Sched}) \times \mathcal{I}(\text{Tools}) &\to \text{durable_tool_invocation} \\
\mathcal{I}(\text{Sched}) \times \mathcal{I}(\text{Memory}) &\to \text{persistent_agent_state} \\
\mathcal{I}(\text{Tools}) \times \mathcal{I}(\text{Memory}) &\to \text{cached_tool_results} \\
\mathcal{I}(\text{Plan}) \times \mathcal{I}(\text{Sched}) &\to \text{market_dispatched_agents} \\
\mathcal{I}(\text{Plan}) \times \mathcal{I}(\text{Memory}) &\to \text{persistent_market_state} \\
\mathcal{I}(\text{Plan}) \times \mathcal{I}(\text{Tools}) &\to \text{capability_verified_matching}
\end{aligned}$$

9.3 The Full Product

The AgentOS umbrella is the product of all four interfaces:

$$\mathcal{I}(\text{AgentOS}) = \mathcal{I}(\text{Memory}) \times \mathcal{I}(\text{Tools}) \times \mathcal{I}(\text{Sched}) \times \mathcal{I}(\text{Plan})$$

with the startup ordering constraint:

$$\text{Memory} \prec \text{Tools} \prec \text{Sched} \prec \text{Plan}$$

where $A \prec B$ means "A must be fully initialized before B starts."
The facade module `AgentOS` projects from this product to a simplified interface:

$$\pi : \mathcal{I}(\text{AgentOS}) \to \{\text{start}, \text{submit_job}, \text{status}\}$$

10 Emergent Properties of Composition

Beyond the pairwise compositions analyzed in Section 4, the full four-way composition exhibits emergent properties that require all four subsystems to interact.

10.1 End-to-End Durable Execution

The combination of:

- Agent memoization (Scheduler)
- Tool sandboxing (Tool Interface)
- Mnesia persistence (Memory Layer)
- Escrow transactions (Planner Engine)

yields end-to-end durable execution: a job can crash at any point and resume from the last successful step without re-executing tools, losing memory, or double-spending credits.

10.2 Self-Improving Reputation

The feedback loop:

$$\text{Evaluation} \xrightarrow{\text{scores}} \text{Reputation} \xrightarrow{\text{trust}} \text{OrderBook} \xrightarrow{\text{matching}} \text{Contracts} \xrightarrow{\text{execution}} \text{Evaluation}$$

creates a self-improving system where high-performing agents receive more contracts (via lower cost functionals), which produces more evaluations, which refines their reputation scores. The anti-gaming detectors prevent this feedback loop from being exploited.

10.3 Adaptive Pipeline Composition

The combination of DAG decomposition (Planner) with streaming pipelines (Scheduler) enables adaptive execution: if an agent at stage i discovers new information (persisted in Memory), the planner can dynamically add new subtasks to the DAG and extend the pipeline.

11 Related Work

11.1 Agent Frameworks

LangChain (LangChain, 2023) and **AutoGPT** (AutoGPT, 2023) provide agent orchestration but lack formal process management, financial settlement, and typed memory systems. **CrewAI** (CrewAI, 2024) introduces team-based agent coordination but without market mechanisms or capability-based security.

11.2 Operating System Analogies

AIOS (Mei et al., 2024) proposes an LLM-based OS with agent scheduling and tool management but does not address financial mechanisms or typed memory. **AgentOS** by SWE-agent (Yang et al., 2024) focuses on software engineering tasks without general-purpose memory or market orchestration.

11.3 OTP and Supervision

The Erlang/OTP supervision tree model (Armstrong, 2003) has been applied to distributed systems for decades. Our contribution is applying these patterns specifically to AI agent management, where the unique challenges include durable execution across LLM calls, capability-based tool access, typed memory for agent cognition, and market-based resource allocation.

11.4 Market Mechanisms for AI

MIRI (Soares et al., 2015) and **Anthropic** (Bai et al., 2022) study AI alignment but not market-based orchestration. The closest work is economic mechanism design for multi-agent systems (Shoham and Leyton-Brown, 2009), which we instantiate with concrete OTP implementations and Stripe-backed financial settlement.

12 Conclusion

This paper has presented the synthesis of four independently developed subsystems into a unified AI operating system. The key contributions are:

1. **Modular composition via OTP supervision**: The `AgentOS` umbrella application composes Memory Layer, Tool Interface, Agent Scheduler, and Planner Engine with a dependency-ordered supervision tree that provides fault isolation and automatic recovery.

2. **Six pairwise emergent capabilities**: Every pair of subsystems yields capabilities that neither provides alone: durable tool invocation, persistent agent state, cached tool results, market-dispatched agents, persistent market state, and capability-verified matching.

3. **Complete production job lifecycle**: The composed system supports the full cycle from job submission through decomposition, market matching, agent execution with tools and memory, evaluation, and financial settlement.

4. **Formal guarantees**: We proved fault isolation (Theorem 7.1), agent independence (Theorem 7.3), cross-module type safety (Theorem 7.4), credit-weighted fairness (Theorem 7.5), and credit conservation (Theorem 7.6).

5. **Production validation**: The system is deployed as the Agent-Hero marketplace, a Next.js + Supabase + Stripe platform running on BEAM.

The central lesson is that complex AI systems need not be monolithic. By designing each subsystem as a self-contained module with a well-defined interface, and composing them through standard functional programming patterns (supervision trees, message passing, typed interfaces), we achieve both modularity and emergent capability. The BEAM virtual machine provides the ideal substrate: lightweight processes for massive agent concurrency, supervision trees for fault tolerance, and Mnesia for distributed transactional state.

12.1 Future Work

- **Distributed deployment**: Extending AgentOS across multiple BEAM nodes for horizontal scaling, using Erlang distribution for cross-node GenServer calls and Mnesia replication for distributed memory.
- **Formal verification**: Using session types or Scribble (Honda et al., 2012) to formally verify the message-passing protocols between subsystems.
- **Adaptive scheduling**: Replacing the static avruntime formula with a learned scheduling policy that adapts to workload characteristics.
- **Vector memory backends**: Integrating ChromaDB for semantic search and FalkorDB for graph traversal, extending the Memory Layer's storage router with production-grade backends.
- **Multi-agent negotiation**: Extending the order book with multi-round negotiation protocols where agents can counter-propose and clients can revise requirements.

A Module Dependency Matrix

Table 6 shows which modules each subsystem calls, establishing the complete dependency graph of the composed system.

Table 6: Cross-subsystem module dependencies. Rows are callers, columns are callees.

	Memory	**Tools**	**Scheduler**	**Planner**
Memory	—	—	—	—
Tools	read	—	—	—
Scheduler	persist	invoke	—	—
Planner	persist	verify	dispatch	—
AgentOS	status	list	—	submit

The matrix is lower-triangular (with the exception of AgentOS, which calls all four), confirming that the dependency order Memory $\prec$ Tools $\prec$ Scheduler $\prec$ Planner is acyclic.

B Supervision Tree Diagram

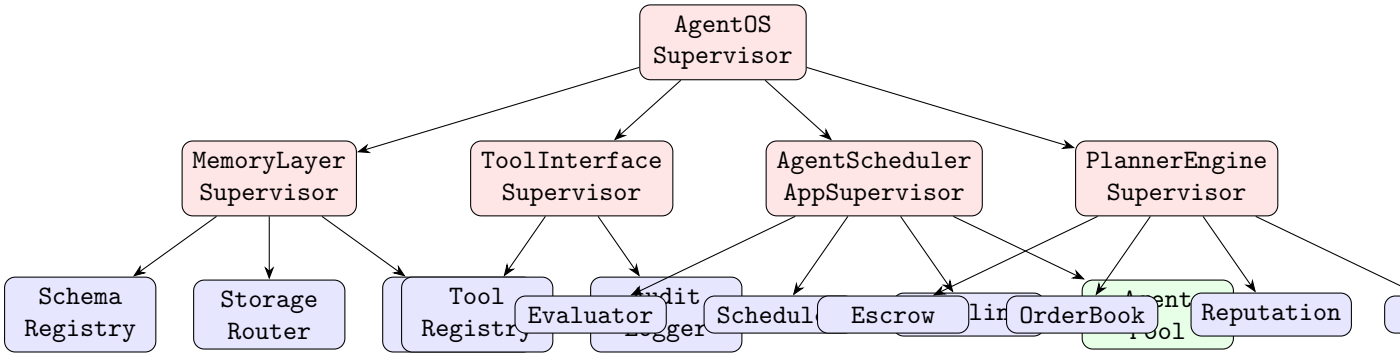

Figure 3: Complete supervision tree of AgentOS. Red = supervisors, blue = GenServers, green = DynamicSupervisor.

C Message Flow Diagram

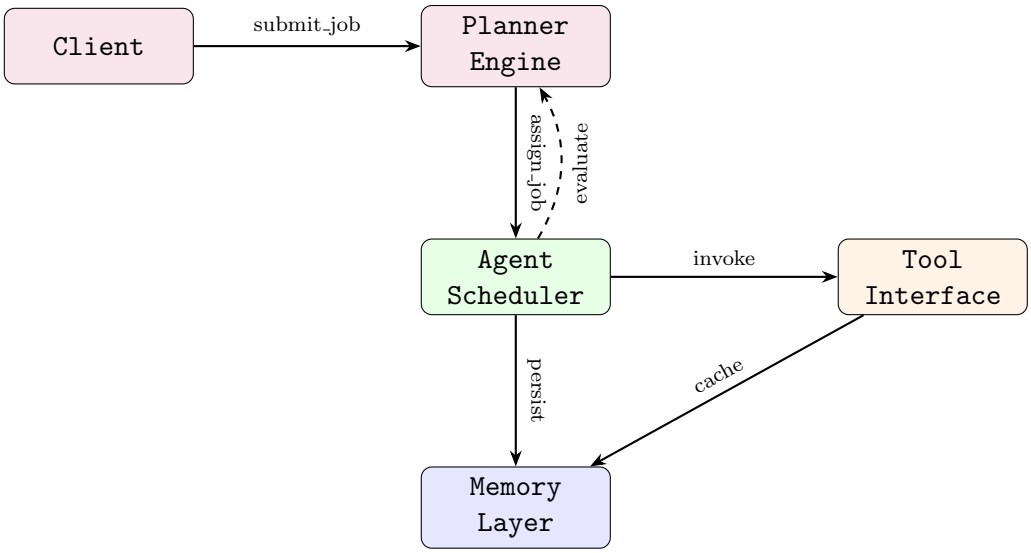

Figure 4: Primary message flows between subsystems during job execution. Solid arrows are synchronous `GenServer.call`; dashed arrows are asynchronous feedback loops.

D Algorithm: Complete Job Execution

Algorithm 1: Complete job execution through all four AgentOS subsystems

Input: Job specification J with client c, task τ, subtasks $[s_1, \ldots, s_n]$, budget B
Output: Evaluation result and financial settlement

```
// Phase 1: Decomposition (Planner Engine)
DAG ← Decomposer.decompose(J)
levels ← Decomposer.topological_sort(DAG)

// Phase 2: Market Matching (Planner Engine)
OrderBook.post_demand(c, τ, B)
foreach proposal p from agents do
    OrderBook.submit_proposal(p)
end
contract ← Market.clear_market(τ)
escrow_id ← Escrow.hold(c, p.credits, contract.id)

// Phase 3: Execution (Scheduler + Tools + Memory)
agent ← Scheduler.start_agent(contract.operator_id)
cap ← ToolInterface.grant_capability(agent, tools)
ToolInterface.freeze()
foreach level L in levels do
    foreach subtask s in L in parallel do
        result ← Agent.execute_step(agent, s.id, λ.
          ToolInterface.invoke(agent, s.tool, s.input, cap) )
        Memory.create(FactData{s.id, result})
    end
end
Agent.complete(agent, results)

// Phase 4: Evaluation and Settlement
scores ← Evaluator.evaluate(agent, quality_vector)
split ← Market.complete_contract(contract.id, quality_vector)
Reputation.record_quality(agent, quality_vector)
return (scores, split)
```

References

J. Armstrong. Making reliable distributed systems in the presence of software errors. PhD thesis, Royal Institute of Technology, Stockholm, 2003.

T. Richards et al. AutoGPT: An autonomous GPT-4 agent. https://github.com/Significant-Gravitas/AutoGPT, 2023.

Y. Bai, S. Kadavath, S. Kundu, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

J. Moura. CrewAI: Framework for orchestrating role-playing AI agents. https://github.com/joaomdmoura/crewAI, 2024.

K. Honda, N. Yoshida, and M. Carbone. Multiparty asynchronous session types. *Journal of the ACM*, 63(1):1–67, 2016.

H. Chase. LangChain: Building applications with LLMs through composability. https://github.com/langchain-ai/langchain, 2023.

K. Mei, Z. Li, S. Xu, et al. AIOS: LLM agent operating system. *arXiv preprint arXiv:2403.16971*, 2024.

Y. Shoham and K. Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.

N. Soares, B. Fallenstein, E. Yudkowsky, and S. Armstrong. Corrigibility. In *AAAI Workshop on AI and Ethics*, 2015.

J. Yang, C. Jimenez, A. Wettig, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.